Containers and Kubernetes

Study notes for a cloud infrastructure course.

Topics: containers, Docker images, Kubernetes objects, and running machine learning models in production (MLOps).

Containers vs virtual machines

A virtual machine runs a full guest operating system on top of a hypervisor. Each VM has its own kernel, so it is heavy: gigabytes of disk and slow to boot.

A container shares the host kernel and isolates only the process, using Linux namespaces (what the process can see) and cgroups (how much CPU and memory it can use).

Containers start in seconds and are small, so many more fit on one server. The trade-off is weaker isolation, because all containers share one kernel.

Docker images and layers

An image is a read-only template made of stacked layers. Each instruction in a Dockerfile that changes the filesystem creates a new layer.

Layers are cached. If a layer has not changed, Docker reuses it on the next build, so instructions that change rarely (installing dependencies) should come before instructions that change often (copying source code).

A container is a running instance of an image with a thin writable layer on top. Data written there is lost when the container is removed, unless a volume is used.

Writing a Dockerfile

FROM picks the base image, for example python:3.12-slim. Slim and alpine images reduce size and attack surface.

WORKDIR sets the working directory. COPY adds files from the build context. RUN executes a command at build time. CMD sets the default command at run time.

A .dockerignore file keeps virtual environments, caches and secrets out of the build context. Run the process as a non-root user with the USER instruction.

Kubernetes architecture

The control plane makes cluster-wide decisions. The API server is the front door for every request. etcd is a consistent key-value store holding all cluster state. The scheduler assigns new pods to nodes. The controller manager runs control loops that move actual state toward desired state.

Each worker node runs the kubelet, which starts containers and reports their health, kube-proxy, which programs network rules for Services, and a container runtime such as containerd.

Pods

A pod is the smallest deployable unit in Kubernetes. It holds one or more containers that share a network namespace (one IP address) and can share volumes.

Pods are ephemeral: if a node dies, its pods are not moved, they are replaced by new pods with new IP addresses. That is why clients should not talk to pod IPs directly.

Init containers run to completion before the main containers start, for example to download data or wait for a dependency.

Deployments and rolling updates

A Deployment declares how many replicas of a pod should run. It manages a ReplicaSet, which creates or deletes pods to keep the count correct.

Changing the pod template (for example a new image tag) triggers a rolling update: new pods are started and old ones removed a few at a time, controlled by maxSurge and maxUnavailable, so the service stays up.

If the new version is broken, kubectl rollout undo returns to the previous ReplicaSet.

Services

A Service gives a stable name and virtual IP to a changing set of pods, selected by labels. Cluster DNS resolves the Service name, so pods find each other by name.

ClusterIP is the default type and is reachable only inside the cluster. NodePort opens the same port on every node. LoadBalancer asks the cloud provider for an external load balancer. An Ingress routes HTTP traffic by host name and path.

ConfigMaps and Secrets

Configuration should live outside the image so the same image runs in every environment. A ConfigMap stores non-sensitive settings as key-value pairs.

A Secret stores sensitive values such as passwords and API tokens. By default Secrets are only base64 encoded, not encrypted, so enable encryption at rest in etcd and restrict access with RBAC.

Both can be exposed to a pod as environment variables or as files in a volume.

Persistent storage

Container filesystems disappear with the pod. Stateful data needs a volume.

A PersistentVolume (PV) is a piece of storage in the cluster. A PersistentVolumeClaim (PVC) is a request for storage by a pod, with a size and an access mode such as ReadWriteOnce.

A StorageClass lets claims create volumes on demand (dynamic provisioning). StatefulSets give each replica its own stable claim and network identity.

Health checks: probes

A liveness probe checks whether the container is still working. If it fails, the kubelet restarts the container, which fixes deadlocks.

A readiness probe checks whether the container can accept traffic. If it fails, the pod is removed from Service endpoints but not restarted, which is useful while a model is still loading.

A startup probe protects slow-starting applications by disabling the other probes until the application has started once.

Resource requests and limits

A request is the amount of CPU or memory the scheduler reserves for a container; a pod is only placed on a node with enough unreserved capacity.

A limit is the maximum. A container that exceeds its CPU limit is throttled. A container that exceeds its memory limit is killed and reported as OOMKilled.

Machine learning models are memory hungry, so set memory requests close to the real usage measured under load.

Autoscaling

The Horizontal Pod Autoscaler (HPA) changes the number of replicas of a Deployment based on a metric, most often average CPU utilization compared with the request.

The Vertical Pod Autoscaler adjusts requests instead of replica count. The Cluster Autoscaler adds or removes nodes when pods cannot be scheduled.

Autoscaling needs the metrics-server add-on to read pod CPU and memory usage.

MLOps basics

MLOps applies DevOps practice to machine learning: version the code, data and model, test automatically, and deploy through a pipeline instead of by hand.

Model serving exposes a model behind an API. Keep the model and its dependencies in a container so training and production use the same environment.

After deployment, monitor latency, errors and prediction quality. Data drift means the input data in production no longer looks like the training data, so quality drops and the model may need retraining. An evaluation set run in CI catches regressions before release.